



当我们用一个派生类对象为一个基类对象初始化或赋值时，只有该派生类对象中的基类部分会被拷贝、移动或赋值，它的派生类部分将被忽略掉。

15.2.3 节练习

练习 15.8: 给出静态类型和动态类型的定义。

练习 15.9: 在什么情况下表达式的静态类型可能与动态类型不同？请给出三个静态类型与动态类型不同的例子。

练习 15.10: 回忆我们在 8.1 节（第 279 页）进行的讨论，解释第 284 页中将 ifstream 传递给 Sales_data 的 read 函数的程序是如何工作的。

关键概念：存在继承关系的类型之间的转换规则

要想理解在具有继承关系的类之间发生的类型转换，有三点非常重要：

- 从派生类向基类的类型转换只对指针或引用类型有效。
- 基类向派生类不存在隐式类型转换。
- 和任何其他成员一样，派生类向基类的类型转换也可能会由于访问受限而变得不可行。我们将在 15.5 节（第 544 页）详细介绍可访问性的问题。

尽管自动类型转换只对指针或引用类型有效，但是继承体系中的大多数类仍然（显式或隐式地）定义了拷贝控制成员（参见第 13 章）。因此，我们通常能够将一个派生类对象拷贝、移动或赋值给一个基类对象。不过需要注意的是，这种操作只处理派生类对象的基类部分。



15.3 虚函数

如前所述，在 C++ 语言中，当我们使用基类的引用或指针调用一个虚成员函数时会执行动态绑定（参见 15.1 节，第 527 页）。因为我们直到运行时才能知道到底调用了哪个版本的虚函数，所以所有虚函数都必须有定义。通常情况下，如果我们不使用某个函数，则无须为该函数提供定义（参见 6.1.2 节，第 186 页）。但是我们必须为每一个虚函数都提供定义，而不管它是否被用到了，这是因为连编译器也无法确定到底会使用哪个虚函数。

对虚函数的调用可能在运行时才被解析

当某个虚函数通过指针或引用调用时，编译器产生的代码直到运行时才能确定应该调用哪个版本的函数。被调用的函数是与绑定到指针或引用上的对象的动态类型相匹配的那一个。

举个例子，考虑 15.1 节（第 527 页）的 print_total 函数，该函数通过其名为 item 的参数来进一步调用 net_price，其中 item 的类型是 Quote&。因为 item 是引用而且 net_price 是虚函数，所以我们到底调用 net_price 的哪个版本完全依赖于运行时绑定到 item 的实参的实际（动态）类型：

```
Quote base("0-201-82470-1", 50);
print_total(cout, base, 10);           // 调用 Quote::net_price
Bulk_quote derived("0-201-82470-1", 50, 5, .19);
```

```
print_total(cout, derived, 10);           // 调用 Bulk_quote::net_price
```

在第一条调用语句中, item 绑定到 Quote 类型的对象上, 因此当 print_total 调用 net_price 时, 运行在 Quote 中定义的版本。在第二条调用语句中, item 绑定到 Bulk_quote 类型的对象上, 因此 print_total 调用 Bulk_quote 定义的 net_price。

605

必须要搞清楚的一点是, 动态绑定只有当我们通过指针或引用调用虚函数时才会发生。

```
base = derived;                          // 把 derived 的 Quote 部分拷贝给 base
base.net_price(20);                      // 调用 Quote::net_price
```

当我们通过一个具有普通类型(非引用非指针)的表达式调用虚函数时, 在编译时就会将调用的版本确定下来。例如, 如果我们使用 base 调用 net_price, 则应该运行 net_price 的哪个版本是显而易见的。我们可以改变 base 表示的对象的值(即内容), 但是不会改变该对象的类型。因此, 在编译时该调用就会被解析成 Quote 的 net_price。

关键概念: C++的多态性

OOP 的核心思想是多态性 (polymorphism)。多态性这个词源自希腊语, 其含义是“多种形式”。我们把具有继承关系的多个类型称为多态类型, 因为我们能使用这些类型的“多种形式”而无须在意它们的差异。引用或指针的静态类型与动态类型不同这一事实正是 C++ 语言支持多态性的根本所在。

当我们使用基类的引用或指针调用基类中定义的一个函数时, 我们并不知道该函数真正作用的对象是什么类型, 因为它可能是一个基类的对象也可能是一个派生类的对象。如果该函数是虚函数, 则直到运行时才会决定到底执行哪个版本, 判断的依据是引用或指针所绑定的对象的真实类型。

另一方面, 对非虚函数的调用在编译时进行绑定。类似的, 通过对象进行的函数(虚函数或非虚函数)调用也在编译时绑定。对象的类型是确定不变的, 我们无论如何都不可能令对象的动态类型与静态类型不一致。因此, 通过对象进行的函数调用将在编译时绑定到该对象所属类中的函数版本上。



当且仅当对通过指针或引用调用虚函数时, 才会在运行时解析该调用, 也只有在这种情况下对象的动态类型才有可能与静态类型不同。

派生类中的虚函数

当我们在派生类中覆盖了某个虚函数时, 可以再一次使用 virtual 关键字指出该函数的性质。然而这么做并非必须, 因为一旦某个函数被声明成虚函数, 则在所有派生类中它都是虚函数。

一个派生类的函数如果覆盖了某个继承而来的虚函数, 则它的形参类型必须与被它覆盖的基类函数完全一致。

同样, 派生类中虚函数的返回类型也必须与基类函数匹配。该规则存在一个例外, 当类的虚函数返回类型是类本身的指针或引用时, 上述规则无效。也就是说, 如果 D 由 B 派生得到, 则基类的虚函数可以返回 B* 而派生类的对应函数可以返回 D*, 只不过这样的返回类型要求从 D 到 B 的类型转换是可访问的。15.5 节(第 544 页)将介绍如何确定一个基类的可访问性, 在 15.8.1 节(第 561 页)中我们将看到这种虚函数的一个实际例子。

606



基类中的虚函数在派生类中隐含地也是一个虚函数。当派生类覆盖了某个虚函数时，该函数在基类中的形参必须与派生类中的形参严格匹配。

final 和 override 说明符

如我们将要在 15.6 节 (第 550 页) 介绍的, 派生类如果定义了一个函数与基类中虚函数的名字相同但是形参列表不同, 这仍然是合法的行为。编译器将认为新定义的这个函数与基类中原有的函数是相互独立的。这时, 派生类的函数并没有覆盖掉基类中的版本。就实际的编程习惯而言, 这种声明往往意味着发生了错误, 因为我们可能原本希望派生类能覆盖掉基类中的虚函数, 但是一不小心把形参列表弄错了。

C++
11

要想调试并发现这样的错误显然非常困难。在 C++11 新标准中我们可以使用 `override` 关键字来说明派生类中的虚函数。这么做的好处是在使得程序员的意图更加清晰的同时让编译器可以为我们发现一些错误, 后者在编程实践中显得更加重要。如果我们使用 `override` 标记了某个函数, 但该函数并没有覆盖已存在的虚函数, 此时编译器将报错:

```
struct B {
    virtual void f1(int) const;
    virtual void f2();
    void f3();
};

struct D1 : B {
    void f1(int) const override;    // 正确: f1 与基类中的 f1 匹配
    void f2(int) override;        // 错误: B 没有形如 f2(int) 的函数
    void f3() override;           // 错误: f3 不是虚函数
    void f4() override;           // 错误: B 没有名为 f4 的函数
};
```

在 D1 中, `f1` 的 `override` 说明符是正确的, 因为基类和派生类中的 `f1` 都是 `const` 成员, 并且它们都接受一个 `int` 返回 `void`, 所以 D1 中的 `f1` 正确地覆盖了它从 B 中继承而来的虚函数。

D1 中 `f2` 的声明与 B 中 `f2` 的声明不匹配, 显然 B 中定义的 `f2` 不接受任何参数而 D1 的 `f2` 接受一个 `int`。因为这两个声明不匹配, 所以 D1 的 `f2` 不能覆盖 B 的 `f2`, 它是一个新函数, 仅仅是名字恰好与原来的函数一样而已。因为我们使用 `override` 所表达的意思是我們希望能覆盖基类中的虚函数而实际上并未做到, 所以编译器会报错。

因为只有虚函数才能被覆盖, 所以编译器会拒绝 D1 的 `f3`。该函数不是 B 中的虚函数, 因此它不能被覆盖。类似的, `f4` 的声明也会发生错误, 因为 B 中根本就没有名为 `f4` 的函数。

我们还能把某个函数指定为 `final`, 如果我们已经把函数定义成 `final` 了, 则之后任何尝试覆盖该函数的操作都将引发错误:

```
struct D2 : B {
    // 从 B 继承 f2() 和 f3(), 覆盖 f1(int)
    void f1(int) const final;    // 不允许后续的其他类覆盖 f1(int)
};

struct D3 : D2 {
    void f2();                  // 正确: 覆盖从间接基类 B 继承而来的 f2
    void f1(int) const;         // 错误: D2 已经将 f2 声明成 final
};
```

`final` 和 `override` 说明符出现在形参列表（包括任何 `const` 或引用修饰符）以及尾置返回类型（参见 6.3.3 节，第 206 页）之后。

虚函数与默认实参

和其他函数一样，虚函数也可以拥有默认实参（参见 6.5.1 节，第 211 页）。如果某次函数调用使用了默认实参，则该实参值由本次调用的静态类型决定。

换句话说，如果我们通过基类的引用或指针调用函数，则使用基类中定义的默认实参，即使实际运行的是派生类中的函数版本也是如此。此时，传入派生类函数的将是基类函数定义的默认实参。如果派生类函数依赖不同的实参，则程序结果将与我们的预期不符。

Best
Practices

如果虚函数使用默认实参，则基类和派生类中定义的默认实参最好一致。

回避虚函数的机制

在某些情况下，我们希望对虚函数的调用不要进行动态绑定，而是强迫其执行虚函数的某个特定版本。使用作用域运算符可以实现这一目的，例如下面的代码：

```
// 强行调用基类中定义的函数版本而不管 baseP 的动态类型到底是什么
double undiscounted = baseP->Quote::net_price(42);
```

该代码强行调用 `Quote` 的 `net_price` 函数，而不管 `baseP` 实际指向的对象类型到底是什么。该调用将在编译时完成解析。



通常情况下，只有成员函数（或友元）中的代码才需要使用作用域运算符来回避虚函数的机制。

什么时候我们需要回避虚函数的默认机制呢？通常是当一个派生类的虚函数调用它覆盖的基类的虚函数版本时。在此情况下，基类的版本通常完成继承层次中所有类型都要做的共同任务，而派生类中定义的版本需要执行一些与派生类本身密切相关的操作。



如果一个派生类虚函数需要调用它的基类版本，但是没有使用作用域运算符，则在运行时该调用将被解析为对派生类版本自身的调用，从而导致无限递归。

608

15.3 节练习

练习 15.11: 为你的 `Quote` 类体系添加一个名为 `debug` 的虚函数，令其分别显示每个类的数据成员。

练习 15.12: 有必要将一个成员函数同时声明成 `override` 和 `final` 吗？为什么？

练习 15.13: 给定下面的类，解释每个 `print` 函数的机理：

```
class base {
public:
    string name() { return basename; }
    virtual void print(ostream &os) { os << basename; }
private:
```

```

    string basename;
};
class derived : public base {
public:
    void print(ostream &os) { print(os); os << " " << i; }
private:
    int i;
};

```

在上述代码中存在问题吗？如果有，你该如何修改它？

练习 15.14: 给定上一题中的类以及下面这些对象，说明在运行时调用哪个函数：

```

base bobj;          base *bp1 = &bobj;          base &br1 = bobj;
derived dobj;       base *bp2 = &dobj;          base &br2 = dobj;
(a) bobj.print();   (b) dobj.print();           (c) bp1->name();
(d) bp2->name();    (e) br1.print();           (f) br2.print();

```

15.4 抽象基类

假设我们希望扩展书店程序并令其支持几种不同的折扣策略。除了购买量超过一定数量享受折扣外，我们也可能提供另外一种策略，即购买量不超过某个限额时可以享受折扣，但是一旦超过限额就要按原价支付。或者折扣策略还可能是购买量超过一定数量后购买的全部书籍都享受折扣，否则全都不打折。

上面的每个策略都要求一个购买量的值和一个折扣值。我们可以定义一个新的名为 `Disc_quote` 的类来支持不同的折扣策略，其中 `Disc_quote` 负责保存购买量的值和折扣值。其他的表示某种特定策略的类（如 `Bulk_quote`）将分别继承自 `Disc_quote`，每个派生类通过定义自己的 `net_price` 函数来实现各自的折扣策略。

609

在定义 `Disc_quote` 类之前，首先要确定它的 `net_price` 函数完成什么工作。显然我们的 `Disc_quote` 类与任何特定的折扣策略都无关，因此 `Disc_quote` 类中的 `net_price` 函数是没有实际含义的。

我们可以在 `Disc_quote` 类中不定义新的 `net_price`，此时，`Disc_quote` 将继承 `Quote` 中的 `net_price` 函数。

然而，这样的设计可能导致用户编写出一些无意义的代码。用户可能会创建一个 `Disc_quote` 对象并为其提供购买量和折扣值，如果将该对象传给一个像 `print_total` 这样的函数，则程序将调用 `Quote` 版本的 `net_price`。显然，最终计算出的销售价格并没有考虑我们在创建对象时提供的折扣值，因此上述操作毫无意义。

纯虚函数

认真思考上面描述的情形我们可以发现，关键问题并不仅仅是不知道应该如何定义 `net_price`，而是我们根本就不希望用户创建一个 `Disc_quote` 对象。`Disc_quote` 类表示的是一本打折书籍的通用概念，而非某种具体的折扣策略。

我们可以将 `net_price` 定义成纯虚（pure virtual）函数从而令程序实现我们的设计意图，这样做可以清晰地告诉用户当前这个 `net_price` 函数是没有实际意义的。和普通的虚函数不一样，一个纯虚函数无须定义。我们通过在函数体的位置（即在声明语句